CS 61B Spring 2025

Final

Tuesday, May 13, 2025

Your Name: Solutions (last update: Fri May 21, 11:20 PM), more verbose solutions will be provided May 22

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 8 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	6	7	8	Total
Points:	14	8	10	18	18	7	10	15	100

- ◯ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible**.
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

> I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

_____

_____

_____

_____

_____

Signature: _____

1 Potpourri

(14 Points)

(a) (1 point) A standard Binary Search Tree guarantees logarithmic height regardless of insertion order.

◯ True ● False

Solution: Insertion order matters. For example, if you insert items in order, you'll end up with a spindly tree, i.e. with linear height.

(b) (1 point) True/False: Suppose the counting sort subroutine used by MSD Radix Sort is not stable. MSD Radix Sort will still correctly sort the input array.

● True ◯ False

Solution: Stability isn't necessary. After sorting by the most significant digit, the problem is broken into completely separate subproblems.

(c) (1 point) True/False: Suppose the counting sort subroutine used by LSD Radix Sort is not stable. LSD Radix Sort will still correctly sort the input array.

◯ True ● False

(d) (1 point) True/False: Quicksort with shuffling has a worst-case runtime of $\Theta(N^2)$ when sorting an array of N distinct elements.

◯ True, because it sorts the entire array twice in each recursive call.

◯ True, because it uses extra memory to store copies of the array.

● True, because poor pivot choices can lead to extremely unbalanced partitions.

◯ False, because the shuffling prevents a worst case runtime of $\Theta(N^2)$.

(e) (1 point) What is the state of the array [3, 5, 2, 4, 1] after one pass of selection sort?

● [1, 5, 2, 4, 3] ◯ [2, 3, 5, 4, 1]

◯ [3, 2, 5, 4, 1] ◯ [1, 3, 5, 2, 4]

(f) (1 point) What is the state of the array [17, 23, 45, 12, 34] after the one pass of LSD Radix Sort?

● [12, 23, 34, 45, 17] ◯ [12, 17, 23, 34, 45] ◯ [12, 17, 23, 45, 34]

(g) (1 point) For the array [4,3,2,1], how many element swaps does insertion sort perform?

◯ 3 ◯ 5

◯ 4 ● 6

For parts h and i, consider the following modification to a heap. A d-ary heap, or d-heap, is a generalization of the binary heap where each node can have up to d children, instead of just two. Assume that d is an integer satisfying $2 \leq d \leq N - 1$.

(h) (2 points) What is the worst-case runtime for `removeMin()` of a d-ary heap, assuming the heap contains N elements? Answer in terms of d and N.

○ $\Theta(1)$	○ $\Theta(N)$	○ $\Theta(N \log_d(N))$
○ $\Theta(\log_d(N))$	○ $\Theta(dN)$	○ $\Theta(dN^2)$
○ $\Theta(\log_N(d))$	● $\Theta(d \log_d(N))$	○ $\Theta(N^2)$

(i) (2 points) What is the worst-case runtime for `insert()` of a d-ary heap, assuming the heap contains N elements? Answer in terms of d and N.

○ $\Theta(1)$	○ $\Theta(N)$	○ $\Theta(N \log_d(N))$
● $\Theta(\log_d(N))$	○ $\Theta(dN)$	○ $\Theta(dN^2)$
○ $\Theta(\log_N(d))$	○ $\Theta(d \log_d(N))$	○ $\Theta(N^2)$

For subparts j and k, suppose we insert the following numbers into an empty BST in the following order: [10, 5, 15, 3, 7, 12, 18].

(j) (1 point) Which of the following is the in-order traversal of the BST?

- ● [3, 5, 7, 10, 12, 15, 18]
- ○ [3, 7, 5, 12, 18, 15, 10]
- ○ [3, 5, 7, 15, 12, 18, 10]
- ○ None of the above

(k) (1 point) Which of the following is the post-order traversal of the BST?

- ○ [3, 5, 7, 10, 12, 15, 18]
- ● [3, 7, 5, 12, 18, 15, 10]
- ○ [3, 5, 7, 15, 12, 18, 10]
- ○ None of the above

(l) (1 point) Suppose you are given a directed acyclic graph. Is sorting the vertices in ascending order of in-degree guaranteed to produce a valid topological ordering?

- ○ Yes, because vertices with lower in-degree have fewer dependencies and should appear earlier.
- ○ Yes, because this is the definition of a topological ordering.
- ○ No, because this is only guaranteed if the graph is a tree.
- ● No, because a vertex with a lower in-degree may still depend on a vertex with a higher in-degree.

2 PoTwourri

(8 Points)

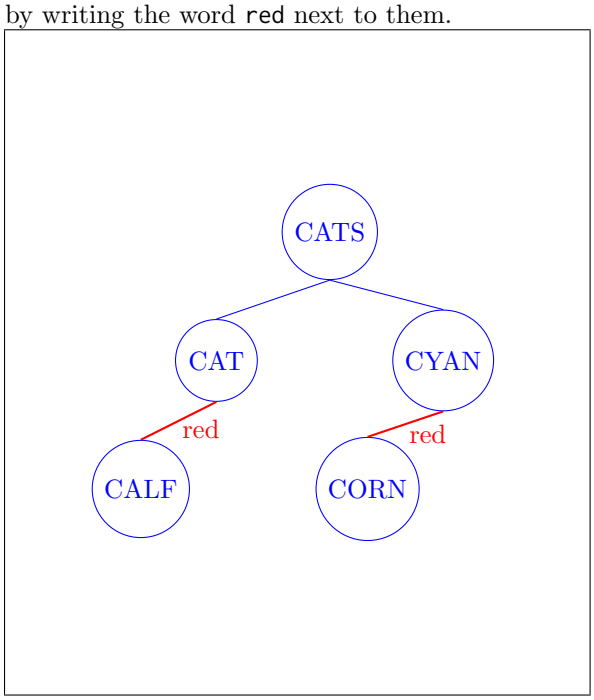
(a) (6 points) Suppose we insert the five words below into a set. Draw the tree that results **after all five words are inserted** if the set is implemented using a **left-leaning red–black tree (LLRB)**, and if the set is implemented using a **Trie**.

Assume the words are inserted in the exact order shown, from left to right. Note: CAT $\prec$ CATS.

CAT	CORN	CATS	CALF	CYAN

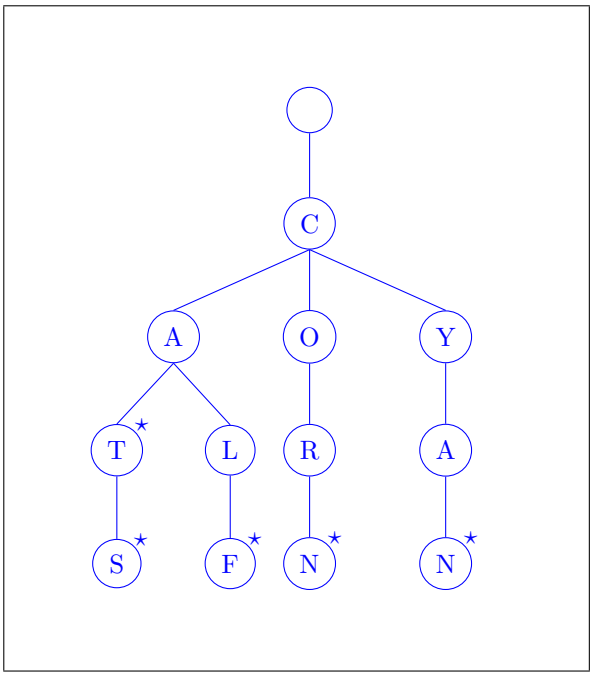
LLRB

Use circles to represent nodes. Indicate red links by writing the word `red` next to them.



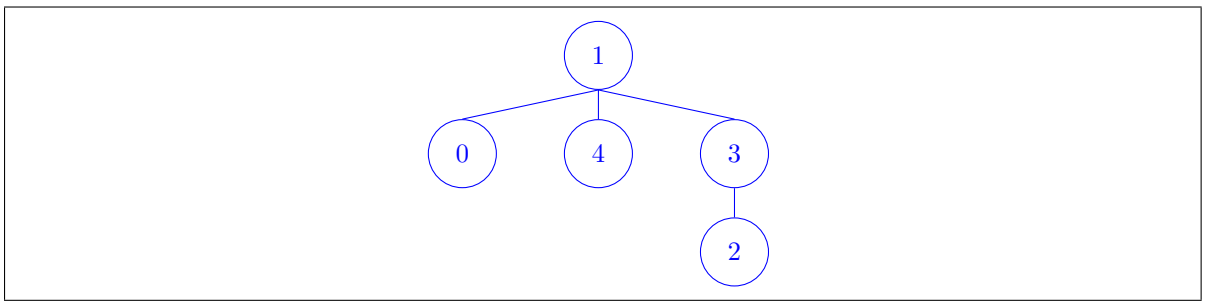
Trie

Use circles to represent nodes. Draw a star next to every node that corresponds to a complete key.



(b) (2 points) Suppose we make the following calls on a new `WeightedQuickUnion` **without path compression** and 5 elements. Draw the resulting tree below. Break ties by linking the first tree below the second. For example, `connect(0, 1)` on a freshly initialized `WeightedQuickUnion` would place the 0 below the 1.

`connect(0,1), connect(2,3), connect(0,4), connect(2,0)`



3 Lost From The Start

(10 Points)

Write the function `findHead`, which receives as input a list of nodes in a non-circular, singly-linked list (in some order), and finds the head of the list. More precisely, `findHead` behaves as follows:

- Input: `List<Node> nodes`, consisting of precisely the Nodes in a singly-linked list, in any order. You may assume that all Nodes have distinct `values`.
- Output: The `Node` that is the head of this singly-linked list (i.e. the node containing the first item of the linked list).
- Runtime: Your code must run in $\Theta(N)$ time, where N is the number of `Nodes` in `nodes`.

Note: `n.next` is `null` if it does not point to another node.

```
public class Node {
    public int value;
    public Node next;
}

public Node findHead(List<Node> nodes) {

    Set<Node> body = new HashSet<>();
    // Note: A Set<Integer> can also work since the integers are unique.

    for (Node n : nodes) {

        if (n.next != null) {

            body.add(n.next);
        }
    }

    for (Node n : nodes) {

        if (!body.contains(n)) {

            return n;
        }
    }

    return null;
}
```

4 Fibonacci Fever

(18 Points)

The Fibonacci sequence is defined as follows:

- $F_0 = 0$
- $F_1 = 1$
- For all integers $N > 1$, $F_N = F_{N-1} + F_{N-2}$

The sequence continues as $0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$, and $F_{12} = 144$.

Since the Fibonacci sequence grows exponentially (at a rate of $\Theta(\phi^N)$, where $\phi \approx 1.618$), competitive programmers often compute the Nth Fibonacci number modulo a prime p instead of its full value. For example, $F_{12} \ \% \ 13 = 1$.

Consider the function `fibmod`(N, p), which returns the Nth Fibonacci number modulo p. Vanessa has implemented four different versions of `fibmod`, and has left the following pseudocode for you to analyze. For each implementation, determine its asymptotic time and space complexity in terms of N and p. You may assume that all basic integer operations take $\Theta(1)$ time, that each implementation produces the correct result, and that p is a prime number. The asymptotic time complexity for the first implementation has been provided for you.

(a) (2 points)
```
public static int fibmod1(int N, int p) {
    if (N <= 1) return N;
    return (fibmod1(N - 1, p) + fibmod1(N - 2, p)) % p;
}
```

Time complexity: $\Theta(\phi^N)$ Space complexity: $\Theta(N)$

(b) (4 points)
```
public static int fibmod2(int N, int p) {
    if (N <= 1) return N;
    int[] arr = new int[N + 1];
    arr[0] = 0;
    arr[1] = 1;
    for (int i = 2; i <= N; i++) {
        arr[i] = (arr[i - 1] + arr[i - 2]) % p;
    }
    return arr[N];
}
```

Time complexity: $\Theta(N)$ Space complexity: $\Theta(N)$

(c) (4 points)

```
public static class Matrix {
    public int size;
    public int[][] data;
    // Returns an error if data is not a square matrix
    public Matrix(int[][] data) {
        // Error checking omitted
        this.data = data;
        this.size = data.length;
    }
}
// Multiplies two matrices of the same size, modulo p.
// If the matrices have a size of k, then this runs in Theta(k^3) time and Theta(k^2) memory
public static Matrix matmulmod(Matrix x, Matrix y, int p) {
   ... // Code omitted
}

// Returns the result of multiplying x with itself exp times (modulo p).
// If x has size k, runs in Theta(log(exp)) calls to matmulmod, and uses Theta(k^2) memory.
public static Matrix matpowmod(Matrix x, int exp, int p) {
    ... //Code omitted
}

public static int fibmod3(int N, int p) {
    // This is known as the Fibonacci matrix.
    // Exponents of this matrix generate the Fibonacci sequence.
    Matrix fibmatrix = new Matrix(new int[][]{{1,1}, {1,0}});
    return matpowmod(fibmatrix, N, p).data[0][1];
}
```

Time complexity: $\Theta(\log(N))$

Space complexity: $\Theta(1)$

(d) (4 points)
```
public static int fibmod4(int N, int p) {
    // The value of the Fibonacci sequence is periodic modulo p for all prime p.
    // That is, for any prime modulus, the values of F_N mod p will eventually repeat.
    // This is known as the Pisano period of p.
    // It can be shown that this period is always <= 4p for prime p.

    // Computes the Pisano period of p
    ArrayList<Integer> vals = new ArrayList<Integer>(List.of(0,1));
    while (vals.get(vals.size() - 1) != 0 || vals.get(vals.size() - 2) != 1) {
        vals.add((vals.get(vals.size() - 1) + vals.get(vals.size() - 2)) % p);
    }
    int pisanoperiod = vals.size() - 1;

    return vals.get(N % pisanoperiod);
}
```

Time complexity: $O(p)$ Space complexity: $O(p)$

(e) (4 points) Vanessa realizes that she actually needs to compute the full value of the Nth Fibonacci number, rather than computing it modulo p. Because the numbers grow exponentially, she can no longer assume that basic math operations take $\Theta(1)$ time. Instead, she must now assume the following:

- The integer A requires $\Theta(\log A)$ memory to store.
- Computing $A + B$ takes $\Theta(\log(A + B))$ time.

Vanessa revisits `fibmod2`, and rewrites it to compute the full Fibonacci number without evaluating modulo p. As a reminder $F_n \in \Theta(\phi^n)$

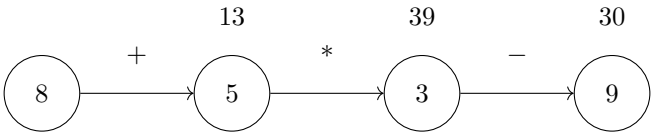
```
public static int fib2(int N) {
    if (N <= 1) return N;
    // BigInts can grow arbitrarily large, unlike normal Java ints
    BigInt[] arr = new BigInt[N + 1];
    arr[0] = new BigInt(0);
    arr[1] = new BigInt(1);
    for (int i = 2; i <= N; i++) {
        arr[i] = BigInt.add(arr[i - 1], arr[i - 2]);
    }
    return arr[N];
}
```

Time complexity: $\Theta(N^2)$ Space complexity: $\Theta(N^2)$

5 Math DAGs!

(18 Points)

Aditya has created a more specific graph representation of short arithmetic calculations called Math DAGs. A Math DAG is a **connected** Math Graph which is also directed and acyclic, where each node contains an integer and each edge contains an arithmetic operation (addition, subtraction, multiplication, or floor division). A Math Path represents a simple path (starting and ending at any node) in a Math Graph or Math DAG (can be a single node as well), and has a value equal to the result of computing the expression created along that path. As a refresher:



The Math Path from 8 to 9 would have a value of $((8+5)*3)-9=30$. Note that the value is computed in path order, and no operation has precedence over another.

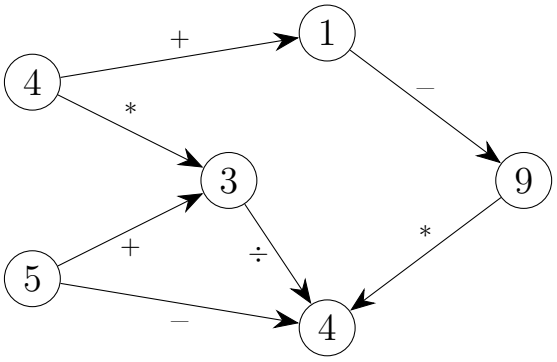
(a) (1 point) True/False: For a Math DAG, $E \log E \in \Theta(E \log V)$.

- ● True
- ○ False

(b) (2 points) For which of the following Math DAGs is the largest-valued path unique? Select all that apply.

- ☐ A Math DAG where each edge is the same operator
- ☐ A Math DAG where each node contains a unique number
- ☐ A Math DAG where the maximum outdegree of any node is 1
- ☐ A Math DAG where the maximum indegree of any node is 1
- ● None of the above

(c) (1 point)



For the above Math DAG, what is the value of the largest-valued Math Path? Write your answer as an integer.

36

Aditya wants to create an algorithm to find the largest value of a Math Path in a Math DAG.

(d) (14 points) Aditya decides initially to assume that a Math DAG will only contain **positive** integers. Complete the `largestMathPath` method below, which returns the numeric value of the largest Math Path in the `DAG`. You may assume that the `MathDAG` is nonempty. Your code must run in $O(V + E)$ time, where V and E are the number of vertices and edges in the MathDAG, respectively.

```
public class MathDAG {
    public class Node {
        public int value;
        public List<GraphEdge> neighbors;
        ...
    }
    public class GraphEdge {
        public Node to;
        public char operator; //Guaranteed to be one of '+', '-', '*', '/'
        ...
    }
    ...
    private List<Node> vertices;
    // Returns a op b, where op is one of '+', '-', '*', and '/'
    public int applyOp(int a, int b, char op) {
        //Implementation omitted
    }
    // Returns a topological sort of this Math DAG
    public List<Node> toposort() {
        //Implementation omitted
    }
    public int findLargestMathPath() {
        Map<Node, Integer> best = new HashMap<>();
        for (__1__ : __2__) {
            ___3__(__4__, __5__);
        }
        for (__6__ : __7__) {
            for (__8__ : __9__) {
               __10__(__11__, __12__(__13__(__14__), __15__(__16__(__17__), __18__, __19__)));
            }
        }
        int globalLargest = __20__;
        for (__21__ : __22__) {
            if (__23__ < __24__(__25__)) {
                __26__ = __27__(__28__);
            }
        }
        return globalLargest;
    }
}
```

For each code blank, fill in the corresponding statement such that the above algorithm works to find the value of the largest-valued Math Path of a `MathDAG` with only positive integers. Some options may be used more than once, or not at all. **If multiple answers can fit in a blank, select the option that minimizes runtime**.

You will receive +0.5 points for a correct response, and -0.1 points for an incorrect response in this section (capped at 0 and 14 points)

A) `Node n`
B) `n`
C) `n.value`
D) `n.neighbors`
E) `GraphEdge e`
F) `e`
G) `e.to`
H) `e.to.value`
I) `e.operator`
J) `Integer.MIN_VALUE`
K) `Integer.MAX_VALUE`
L) `best`
M) `best.get`
N) `best.put`
O) `globalLargest`
P) `Math.max`
Q) `Math.min`
R) `applyOp`
S) `this`
T) `this.toposort()`
U) `this.vertices`

1. A	8. E	15. R	22. U
2. U	9. D	16. M	23. O
3. N	10. N	17. B	24. M
4. B	11. G	18. H	25. B
5. C	12. P	19. I	26. O
6. A	13. M	20. J	27. M
7. T	14. G	21. A	28. B

Solution: Due to an error, the word "connected" was omitted from the definition of a Math DAG; this was clarified late in the exam, and this one-word omission would change the answer to part 5a. As such, 5a was dropped.

6 2D Hashing

(7 Points)

Karen and Peyrin each have their own hash function, and they cannot decide which one to use. They decide to use both functions and create a 2D Hashmap with N rows and M columns. When inserting an item in this Hashmap, Karen's hash function is used to determine which row the item is inserted into, and Peyrin's hash function is used to determine which column the item is inserted into.

For example, to insert element `x` into the hashmap, we compute `int k = x.KarenHash(); int p = x.PeyrinHash();` and put the new item into `buckets[k % N][p % M]`.

Two valid hash functions f and g are **independent** if their values are uncorrelated; that is, knowing the value of $f(x)$ tells us no information about the value of $g(x)$, and vice versa.

The `Dog` class follows as below:

```
public class Dog {
    private final int birthYear;
    private final String name;
    private int weight;

    public Dog(int birthYear, String name, int weight) {
        this.birthYear = birthYear;
        this.name = name;
        this.weight = weight;
    }

    public void setWeight(int weight) { this.weight = weight; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Dog)) return false;
        Dog otherDog = (Dog) o;
        return (this.birthYear == otherDog.birthYear) &&
               (this.name != null && this.name.equals(otherDog.name));
    }

    //Implemented differently in each part
    public int KarenHash() {...}

    public int PeyrinHash(){...}
}
```

(a) (1 point) Given the following `Dog` objects:

```
dog1 = new Dog(2018, "Maxine", 65);
dog2 = new Dog(2018, "Max", 70);
dog3 = new Dog(2020, "Buddy", 60);
```

If `KarenHash` returns `birthYear` and `PeyrinHash` returns `name.length()`, which pairs of dogs would hash to the same bucket in a 2D hashmap, where $N = 10$ and $M = 3$?

- ■ `dog1` and `dog2`
- ☐ `dog1` and `dog3`
- ☐ `dog2` and `dog3`
- ◯ None of the dogs would hash to the same bucket

(b) (4 points) For each of the following pairs of hash codes, determine if at least one of them is **Invalid**, if they are both **Valid** but not independent, or if they are both **Valid and Independent**. You may assume that a dog's name, weight, and birth year are statistically independent.

1. `KarenHash: return birthYear; PeyrinHash: return birthYear;`

   ◯ Invalid ● Valid ◯ Valid and Independent

2. `KarenHash: return birthYear; PeyrinHash: return name.length();`

   ◯ Invalid ◯ Valid ● Valid and Independent

3. `KarenHash: return birthYear + name.length(); PeyrinHash: return birthYear + name.length();`

   ◯ Invalid ● Valid ◯ Valid and Independent

4. `KarenHash: return birthYear; PeyrinHash: return weight;`

   ● Invalid ◯ Valid ◯ Valid and Independent

(c) (1 point) Karen and Peyrin accidentally choose the same (valid, well-distributed) hash function. If $N = 4$ and $M = 6$, what is the most nonempty bins the hashMap can contain?

12

(d) (1 point) Karen was thinking about changing her dog's name, so we need to make the `name` member of the `Dog` class mutable. This would require us to edit the declaration of `name` to `private String name` (removing the `final` keyword), and include a `changeName()` method.

Karen wants `KarenHash` to return `name.length()`. Is this still a valid hash function?

- ● Yes; it remains consistent with `equals()`, so it is valid even if name is mutable.
- ◯ Yes; as long as we don't override `equals()`, the hashcode method does not matter.
- ◯ No, because `name` (and therefore `name.length()`) can change after insertion.
- ◯ No, because the hash function is no longer consistent with `equals()`.

This page intentionally left (mostly) blank.

The exam continues on the next page.

7 Find Bee Leader

(10 Points)

Dylan's backyard is home to a vast network of bee hives. The bee network can be modeled as a connected, undirected graph G with vertices V and edges E, where:

- Each vertex $v \in V$ represents a **hive**.
- Each edge $e \in E$ represents a possible **pollen trail** between two hives, with a traversal cost d (based on wind, distance, and predators).

A subset $Q \subseteq V$ of the hives are designated as **Queen Hives**. Each hive must be connected to **exactly one** Queen Hive using the pollen trails, and no hive should be able to reach more than one Queen Hive to prevent in-fighting.

Design an efficient algorithm that receives as input a graph $G = (V, E)$ and Q, and returns a set of edges such that:

- Every hive is in the connected component of exactly one Queen Hive.
- No two Queen Hives are in the same connected component.
- The total pollen trail cost is **minimized**. You may assume that all pollen costs are positive integers.

You may use without explanation any algorithm or data structure taught in CS61B. Algorithms which asymptotically exceed the runtime bound of the staff solution will receive no credit.

Solution: See next page

Multiple solutions exist. Two example solutions are provided below:

1. Create a dummy node connected to all queens with edges of weight 0. Then run any MST algorithm.

   This runs in $\Theta(E \log V)$ time if we use Prim's algorithm, but can be improved to $O(E\alpha(E))$ or an unknown but provably optimal runtime using Chazelle or Pettie-Ramachandran's improved MST algorithms (discussed briefly in Lecture 25's extra slides).

2. Run Kruskal's algorithm, but before removing the first edge from the Priority Queue, connect all Queens in the internal disjoint set.

   This runs in $\Theta(E \log E)$ time.

Any solution that runs in $O(E \log E)$ received credit for this question. If your algorithm took longer than $O(E \log E)$, we graded it considering only the first $E \log E$ steps.

8 Rebuild Your Own World

(15 Points)

You're refactoring your Project 3 implementation into a video game project. The game consists of multiple customizable levels, each containing a generated world, enemies, and coins. The player collects all coins while avoiding enemies. In addition, you plan to implement saving/loading and visibility radius features.

You've made several design decisions, but some may be inefficient or inflexible. For each of the following design choices, briefly explain:

1. Why the current approach is inefficient, either in runtime, memory usage, or future development flexibility.
2. How you would improve the approach, using better software engineering practices (e.g., design patterns, data structures, modularity).

(a) (3 points) Each level of your world is stored in a separate Java file, where each Java file contains the same code, but with different constants controlling the number of rooms, number of coins, etc. When you want to make a new level, you copy and paste this code to a new file and change the constants appropriately.

(1) Why is the above approach inefficient?

(2) How would you redesign it?

Solution: Repeatedly copying the same code makes the code more inflexible, increases the size of the source code, and makes it harder to make changes to the base algorithm.

A better approach would be to rewrite the world generation algorithm as a function which takes in as input the number of rooms, number of coins, etc., and to have each level call the function with the appropriate parameters when loading.

An even better approach would be to have a single Level class (instead of one class per level), and to store each level's parameters in a text file. In addition to reducing the cost of adding new levels to the game, this has the added benefit of making it easier for non-programmers to make new levels in the future (they just have to change values in a config file instead of making a new class).

(b) (3 points) When the player walks over a coin, the coin is removed. After each move, the game scans every tile to see if any coins remain. If there are 0 coins left, the level ends.

(1) Why is the above approach inefficient?

(2) How would you redesign it?

Solution: This checks all squares on the grid repeatedly for coins, which has a high runtime cost.

A better approach would be to have a variable `int coinsRemaining`, which counts down each time a coin is collected; when the count hits zero, the level ends.

(c) (3 points) You implemented an `Enemy` class with a `String name`, `int x` and `y` coordinates, and a `move()` method. The `move()` method uses a `switch(name)` statement to determine behavior (e.g., `"crab"` moves left-right, `"ghost"` moves through walls). The game state contains a `List<Enemy>` called `enemies`. Every time the player moves, the game iterates over all enemies and determines the new position of each enemy by calling `move()` on them.

(1) Why is the above approach inefficient?

(2) How would you redesign it?

Solution: Putting all "move" operations into a single file makes an extremely bloated file which is easy to accidentally break; changes to one enemy's movement can possibly affect another enemy's movement if you're not careful/accidentally change the wrong line of code.

A better approach would be to define each Enemy as a subclass of the main class Enemy, and define their `move` methods in separate files.

Alternate correct inefficiency claim: If you have a huge number of **enemy types**, a switch statement will have a runtime proportional to the number of enemy types.

Alternate correct inefficiency claim: The design is inflexible if you decide to later allow asynchronous movement.

Common incorrect inefficiency claim: Enemies only move when the player moves. Note: This is not an example of inefficiency. This is just the way the game works. To get credit, you need to explicitly state that this is an inflexible design (or similar).

Common incorrect inefficiency claim: It is inefficient to iterate over the entire list of enemies, as this has runtime proportional to the number of enemies.

Common incorrect redesign: Each enemy should have their own move method. This is not enough information.

(d) (3 points) To save the game, you store the full history of player keypresses in a text file. To load, the game replays the entire keypress sequence to reconstruct the game state.

(1) Why is the above approach inefficient?

(2) How would you redesign it?

Solution: If the player plays an extremely long sequence of moves, then the game needs to store all moves to save the state. In addition, if the player moves many times and simulating moves is slow, loading a file may take too long.

A better approach would be to find the parameters that define the current game state (such as the player's location, enemy locations, current RNG state), and save just that in a file. Various libraries exist to convert a class to a saveable format (JSON may be sufficient for many purposes), but more complicated games will likely need specialized saving/loading features to account for all variables and prevent the user from "hacking in" increased stats.

Common incorrect inefficiency claims: "The runtime is slow", but no further details. Not being able to handle new types of key presses. Some key presses are not useful (e.g. walking into a wall or key presses that are not defined). Won't work if you change the game because the saved key presses won't work anymore (i.e. your save file compatibility will break).

(e) (3 points) To determine if a tile is visible to the player, the game runs the A* algorithm from the player's position to every other tile, using only walkable tiles and a Euclidean distance heuristic. A tile is marked visible if the computed distance is 10 or less. This process is repeated for every tile on the board every time the player moves, updating visibility accordingly.

(1) Why is the above approach inefficient?

(2) How would you redesign it?

Solution: A* is efficient for finding the shortest path from one tile to a specific other tile, but running A* in this way checks all tiles, instead of only the tiles closest to the player.

A better approach would be to use Dijkstra's algorithm, as we want to get the shortest path from the player to every other tile, and then cut the algorithm off when exceeding 10 units of distance from the player. Even better, the grid graph is simple enough that a BFS would also get the correct set of tiles, while reducing the runtime of the algorithm even further.

Nothing on this page is worth any points.

Luck of the Draw

(0 Points)

Write a mathematical expression that non-obviously evaluates to the five-digit number (starting with zero) written on the bottom-right corner of this page.

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!